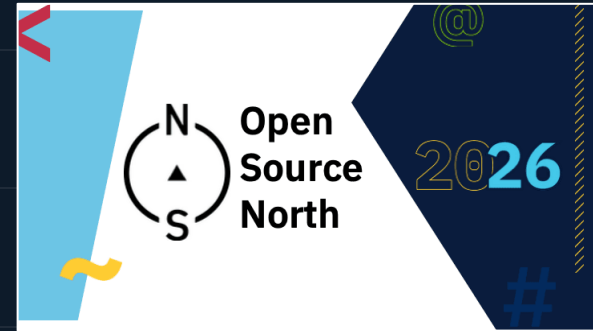


Spec First, Agents Second: Engineering AI Systems with Discipline



A repeatable system for building reliable software with discipline

SYSTEMS-FIRST

Lifecycle Thinking

SPEC-DRIVEN

Development

AI-ASSISTED

Execution

Justin Grammens | Lab651 & Recursive Awesome & Applied AI | Open Source North 2026

Building Software for Regulated Industries

25 Years. Regulated Systems. Real Codebases.



Product-minded & AI assisted engineers · Mobile · Cloud



AI Strategy & Training - Medical · Insurance · Industrial



Non-Profit: Building the next generation of AI Leaders

25 yrs

Building production software, many in regulated industries

IoT + AI

Embedded systems to cloud to LLM pipelines

Educator

Teaching graduate AI/ML Courses at the University of St. Thomas

Thought Leader

Founder + CEO & Thought Leader on Artificial Intelligence

Today's talk is built on production experience — not theory.

2:47 AM. Room 412. Insulin overdue.

The alert system shows no overdue alerts. The code looks clean. Tests pass.

But a developer, six weeks ago, copy-pasted a date comparison. Timezone handling is wrong for daylight saving time. There is no requirement that specified UTC. No test covering timezone edge cases. No audit log to surface the discrepancy.

Just a patient in need of overdue insulin.

The root cause wasn't the developer. It wasn't the timezone. It was the absence of structure.

No requirement captured the constraint. No test verified it. No trace connected requirement to code.

AI Amplifies What's Already There

"VIBE CODING" — Fast, Fragile, Untraceable

- ✗ No explicit requirements → intent lives only in developer's head
- ✗ AI generates confident-looking code with no verified behavior
- ✗ Tests validate the implementation, not the requirement
- ✗ Zero traceability: requirement → design → code → test
- ✗ Audit trail: nonexistent

SPEED VS. RELIABILITY · 2026 DATA



As AI adoption hit the mainstream,
individual code output rose
measurably.

Source: CodeRabbit, State of AI vs. Human Code Generation Report (Dec 2025) —
analysis of 470 open-source pull requests (320 AI-coauthored, 150 human-only).



10.83 issues per AI PR vs. 6.45 in human PRs
across 470 PRs. AI code is 2.25x more likely
to have logic errors, ~2x more security issues.

“
Imagine a junior engineer and you just say: ‘refactor this.’ That’s a huge challenge — they need context, guardrails, rules. That’s exactly how I think about working with AI.

In regulated industries, speed without structure is not a feature. It is a liability.

Speed Goes Up. Quality Quietly Collapses.

GITCLEAR · 211 MILLION LINES OF CODE · 2020 - 2024

REFACTORING (HEALTHY CHANGES)



-60%

25% → under 10%

Refactoring's share of code changes more than halved between 2021 and 2024.

Engineers stopped reshaping existing code.
They added new lines on top of it.

DUPLICATED CODE BLOCKS



8×

Copy-pasted code: 8.3% → 12.3%

Eightfold rise in duplicated blocks —
the first year copy-paste outpaced reuse.

DRY violations multiply maintenance cost
and propagate the same bug to every clone.

CODE CHURN (REWRITTEN < 2 WEEKS)



2×

3.1% (2020) → 5.7%+ (2024)

Code rewritten or reverted within two
weeks nearly doubled — a leading defect signal.

More "almost right" code. More rework.
Productivity gain absorbed by repair cost.

Vulnerabilities Double in One Year

APIIRO · FORTUNE 50 ENTERPRISE STUDY · DEC 2024 – JUN 2025

10×

surge in security findings

Monthly findings climbed from a baseline in December 2024 to over 10,000 per month by June 2025 — a 10× increase in just six months.

WHERE THE RISK IS COMPOUNDING

+322%

Privilege-escalation paths

Auth checks without authorization checks.

+153%

Architectural design flaws

10–100× more expensive to remediate than bugs.

2.74×

More vulnerabilities per 1,000 LOC

Veracode tested 100+ LLMs · 45% security failure rate.

+40%

Repos with exposed secrets

Hardcoded credentials, API keys, tokens.

More Code Shipped This Quarter Than All of Last Year

GITHUB PLATFORM DATA · CONFIRMED BY COO KYLE DAIGLE · 2026



more commits than all of last year

GitHub absorbed ~1 billion commits in all of 2025. The platform is now on pace for 14 billion in 2026 — 275 million per week.

WHAT'S DRIVING THE SURGE

25x

Claude Code commits in 6 months

100K → 2.5M public commits per week.

4x

AI-agent pull requests since Sept '25

4M → 17M+ PRs/month opened by autonomous agents.

~4%

of all public commits — Claude Code alone

Projected to hit 20%+ of daily commits by year-end.

2.1B

CI/CD compute minutes in a single week

Up from 500M in 2023 — agents iterate at machine speed.

Three Practices. One Disciplined Engineering Model.

01

SYSTEMS-FIRST ENGINEERING

Connect every decision to the full system lifecycle — from requirement to design to verification to post-deployment evolution.

- ▶ Risk-driven requirements
- ▶ Architectural integrity over time

02

SPEC-DRIVEN DEVELOPMENT

Structure before syntax. The specification is the primary engineering artifact. Code is its deterministic expression.

- ▶ REQ-NNN format requirements
- ▶ Behavior modeled before code

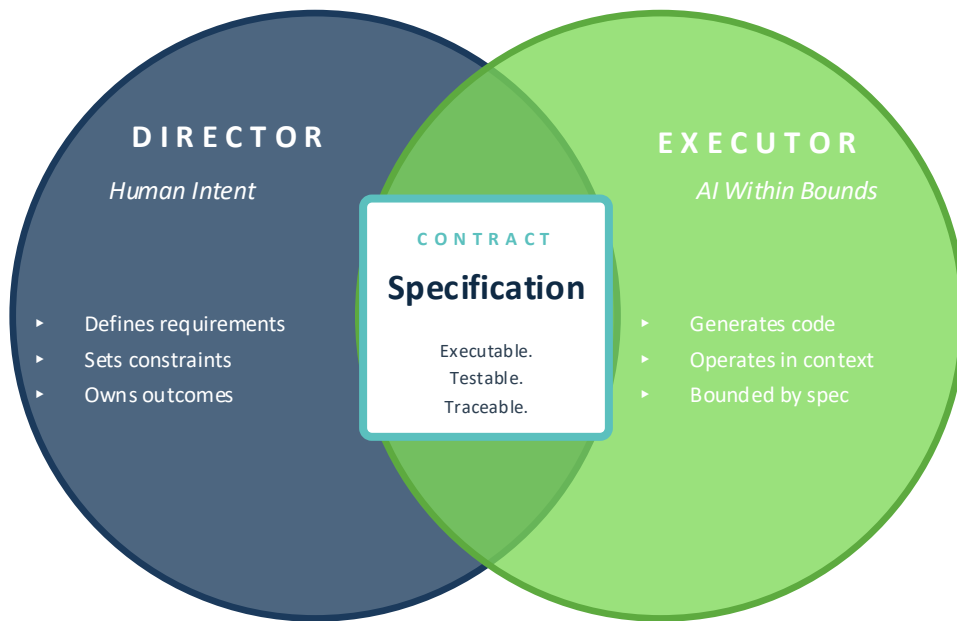
03

AI-ASSISTED EXECUTION

AI operates within defined constraints — not instead of them. The spec is the AI's instruction manual and its safety boundary.

- ▶ Requirement-aware code generation
- ▶ Traced test generation by REQ

Director, Contract, Executor.



The spec is the contract that binds them.

Director (human intent) → **Contract (executable spec)** → Executor (AI within bounds)

Medication Dose Alert Monitor (MDAM)

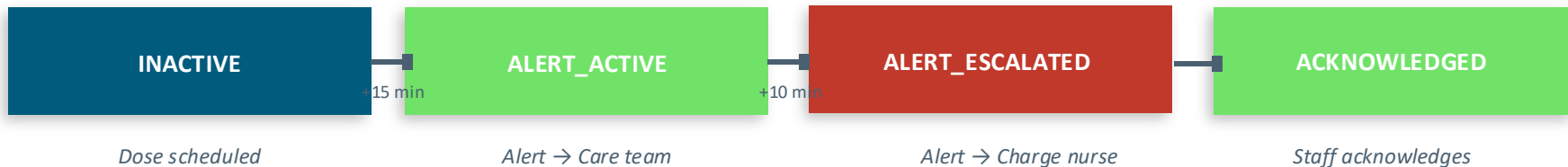
What It Does

Monitors scheduled medication administration. Detects overdue doses.
Generates care alerts with escalation. Requires human acknowledgment.
Maintains an immutable audit log of every state change.

Why This System

- ✓ Timing constraints
- ✓ State transitions
- ✓ Safety-critical behavior
- ✓ Human-in-the-loop
- ✓ Compliance-relevant audit

THE ONE FEATURE: Overdue Dose Alert with Escalation



Every state transition is logged with: prior state · new state · triggering event · UTC timestamp · actor ID

This audit log is immutable. It IS your verification evidence under IEC 62304 and 21 CFR Part 11.

Spec-First: Making Intent Explicit

REQ-001

System SHALL detect any scheduled dose with no administration record within a configurable threshold (default 15 min) and emit an ALERT_ACTIVE event.

REQ-003

System SHALL NOT emit more than one ALERT_ACTIVE event per unique dose event, regardless of evaluation frequency.

REQ-004

System SHALL emit an ALERT_ESCALATED event if an ALERT_ACTIVE event remains unacknowledged for a configurable escalation period (default 10 min).

REQ-006

System SHALL record all state transitions with: prior state, new state, event, UTC timestamp, actor ID. Audit log SHALL be immutable.

REQ-009

On any internal error during detection or escalation, the system SHALL surface a SYSTEM_ALERT to the operator and SHALL NOT suppress or swallow the error silently.

REQ-010

All detection and escalation functions SHALL be deterministic: identical inputs SHALL always produce identical outputs with no side effects.

These requirements are AI-reviewable, AI-implementable, and AI-traceable — because they are explicit and structured.

Same AI. Completely Different Engineering.

✗ VIBE CODING PROMPT

```
"Build me a medication alert system that
notifies nurses when doses are late"
```

What gets generated:

- setTimeout-based timing (no UTC, no injection)
- Hardcoded email function, no role map
- No state machine — flag variable
- No audit log, no actor tracking

Vibe coding has a place: exploration, research, quick bug hunts.

✓ SPEC-DRIVEN PROMPT

```
"Implement DoseStateEngine satisfying REQ-001 through
REQ-012. Pure function. Injected timestamps. JSDoc
tracing each method to requirement ID."
```

What gets generated:

- ✓ Pure function — no hidden state
- ✓ Injected timestamps (deterministic, testable)
- ✓ Explicit state machine — all transitions typed
- ✓ Immutable audit entry on every transition
- ✓ JSDoc: @satisfies REQ-001 per method
- ✓ Tests labeled per requirement, boundary cases covered

Spec-driven is non-negotiable when traceability is required.

The Industry Is Independently Arriving at the Same Answer

AWS / KIRO

Spec-Driven Mode
*Built spec-first workflow
into flagship AI IDE*

GitHub SpecKit

Open Source Framework
*Spec-driven toolkit for
GitHub Copilot & Claude*

SPEC DRIVEN AI ENGINEERING

Research

METR · CodeRabbit · arXiv
*1.7× defects confirmed.
Structure reduces errors.*

Anthropic

Claude Code + Plan Mode
*claude.md + skills drives
spec-aware code generation*

GitHub SpecKit: Structure That AI Can Actually Use

What GitHub SpecKit Is

An open source, spec-driven development framework from GitHub, purpose-built for AI-assisted engineering workflows. SpecKit gives AI the structured context and agents it needs to generate code, tests, and traceability artifacts — rather than guessing at intent from a vague prompt.

- ▶ **System Context Block** Named system, version, author, safety classification
- ▶ **REQ-NNN Requirements** Functional, constraint, and safety requirements in SHALL language
- ▶ **Constitution** Engineering invariants that AI cannot violate across the session

LIVE DEMO

We'll now leave the slides.

- ① **Open MDAM project · run: claude**
- ② **Walk the spec artifacts — show what four commands produced**
- ③ **Implement T020 → evaluateDetection() engine**
- ④ **Implement T021 + T022 → 60 tests by REQ**
- ⑤ **Implement T034 → traceability matrix**

GitHub SpecKit is open source — github.com/github/spec-kit · Works with GitHub Copilot and other AI assistants

After the demo we return here to close with the traceability matrix and lifecycle argument.

This Spec Was Built With Four SpecKit Agents

01 /speckit-constitution

Establish non-negotiable engineering principles: IEC 62304, determinism, @satisfies traceability, immutable audit, fail-safe, strict TypeScript.

PROMPT: "Create governing principles for a safety-critical medication alert monitor"

→ .specify/memory/constitution.md

02 /speckit-specify

Describe WHAT and WHY, not the tech stack. 3 user stories, 12 requirements (REQ-001–REQ-012), state machine, success criteria.

PROMPT: "Overdue Dose Alert with Escalation — OBJECTIVE / BEHAVIOR / CONSTRAINTS / VERIFICATION"

→ specs/001-overdue-dose-alert/spec.md

03 /speckit-plan

TypeScript strict mode, pure functions, injected time, annotations, append-only audit. AI makes 7 architecture decisions from spec alone.

PROMPT: "TypeScript strict mode. Node.js 20 LTS. Vitest. PostgreSQL via adapter."

→ plan.md · data-model.md · research.md

04 /speckit-tasks

35 tasks across 6 phases: types → ports → fakes → state engine → tests (by REQ) → traceability matrix. 100% REQ coverage.

PROMPT: "Sequence tasks as: types → state engine → test suite → traceability matrix"

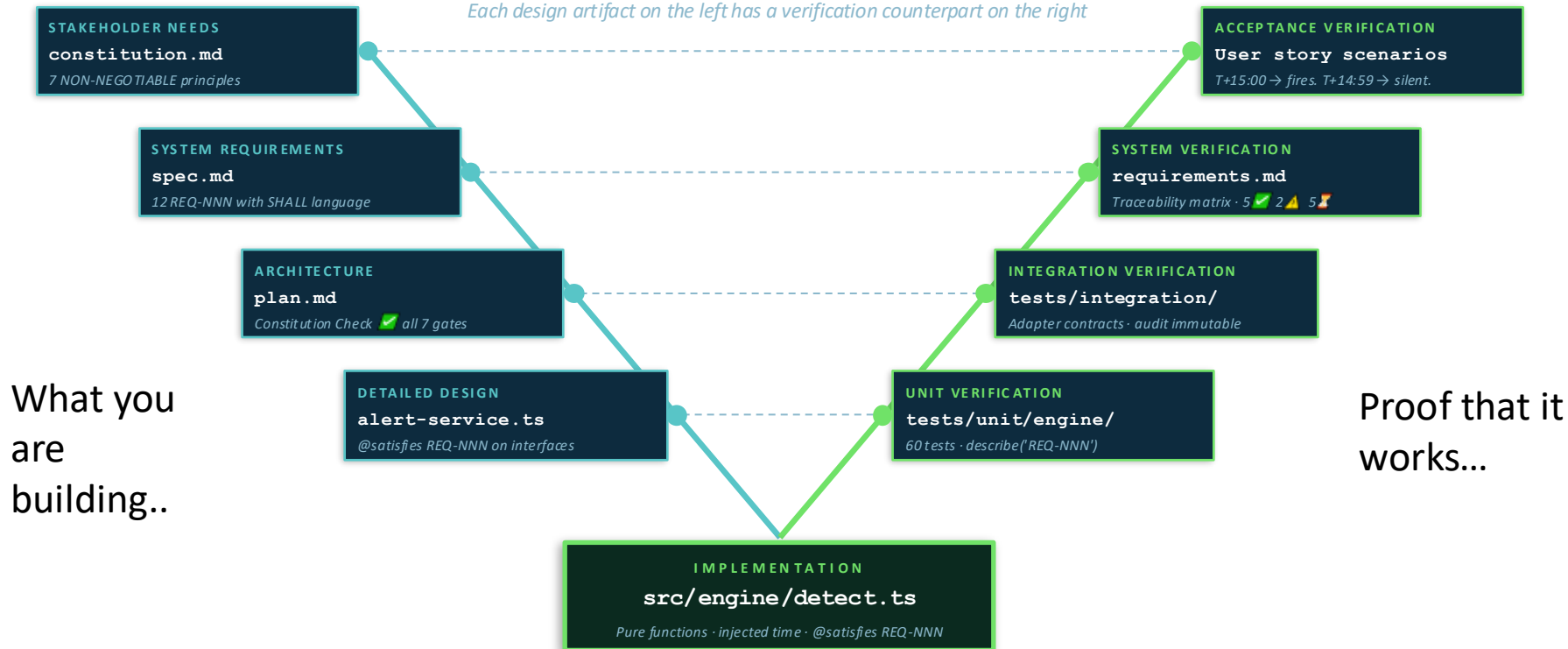
→ specs/001-overdue-dose-alert/tasks.md

These four commands ran before this talk. The files they generated are what you'll see opened on screen during the demo.

Same workflow. Every time. Repeatable by design.

SpecKit Maps Directly to the V-Model

Each design artifact on the left has a verification counterpart on the right



Every left-side artifact has a right-side verification. SpecKit doesn't just produce design — it produces the verification chain that proves design was followed.

Starting Monday: Three Scenarios.

01

NEW PROJECT

Run all four commands before writing a single line of code.

- ▶ specify init → claude → /speckit-constitution → /speckit-specify → /speckit-plan → /speckit-tasks
- ▶ Then: Implement by task ID (e.g. "Implement T020 from tasks.md")
- ▶ Result: spec, plan, tasks, implementation — all traceable from day one

02

EXISTING PROJECT

Pick the next feature. Run the four commands on that feature only.

- ▶ Don't retrofit 200,000 lines — start at the next feature boundary
- ▶ /speckit-constitution runs once per repo · /speckit-specify runs per feature
- ▶ Result: new work is traceable · old work stays as-is · gap closes over time

03

TEAM ADOPTION

Constitution goes in the repo. PR template enforces REQ-NNN.

- ▶ Commit constitution.md to .specify/memory/ — it's a version-controlled artifact
- ▶ Add REQ-NNN field to PR template · CI traceability check fails build if missing
- ▶ Result: every engineer follows spec-first automatically — no manual enforcement

Same four commands. Same prompts. Every time. Repeatable by design.

Three Truths.

- 01 AI doesn't make **bad** engineering **good**. It makes bad engineering **faster**.
 - 02 A spec is not bureaucracy. It's the difference between building **fast** and building **right fast**.
 - 03 Traceability isn't a compliance artifact. It's how you prove — to yourself, your team, and **stake holders** — that you knew what you were doing.
-

